OAVS • TGT RECRUITMENT EXAMINATION

Computer Science Teacher

MCQ Question Bank — Complete Practice Set

Topic: Arrays

150	19	100%
TOTAL MCQS	SUB-TOPICS	ANSWERS + EXPLANATIONS

Strictly aligned to the OAVS TGT Computer Science — Arrays & Collections syllabus

Table of Contents

1. Concept of Collection (8 Qs) 3
2. Types of Collections Supported by C (6 Qs) 6
3. Array Collection and its Features (8 Qs) 9
4. Concept of Indexing (8 Qs) 13
5. Index Variable (6 Qs) 16
6. Index Type (6 Qs) 19
7. Positional Value of a Member of Array Collection (6 Qs) 22
8. Concept of Dimension and Size of an Array (8 Qs) 25
9. C Syntax for Declaration of Array (12 Qs) 28
10. Name of the Array and Its Type (6 Qs) 33
11. Referring Individual Elements (8 Qs) 36
12. Entering Data into an Array (8 Qs) 39
13. Reading Data from an Array (6 Qs) 42
14. Array Initialization (10 Qs) 45
15. List of Initializers (8 Qs) 49
16. Size Option (6 Qs) 52
17. Bounds Checking (8 Qs) 55
18. Two-Dimensional Arrays (16 Qs) 58
19. Similarities Between Dimension and Nesting (6 Qs) 64

Chapter 1: Concept of Collection 8 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q1 | Definition | Concept of Collection

In programming, a "collection" is best defined as:

A. A comment used to document code.

B. A group of related data elements stored and organized so they can be accessed and processed together.

C. A keyword reserved for declaring constants.

D. A single variable holding one primitive value.

Correct Answer: B

Explanation: A collection groups multiple related elements under one identifier so they can be handled as a unit.

Q2 | Conceptual | Concept of Collection

What is the main purpose of using a collection in a program?

A. To make the program run without a compiler.

B. To prevent the use of loops.

C. To store only floating-point constants.

D. To store and manage multiple related values efficiently under one name.

Correct Answer: D

Explanation: Collections exist so that many related values can be managed together instead of individually.

Q3 | Statement-based | Concept of Collection

Statement: "Collections become necessary when a program must handle many related values that would otherwise require separate variables." Evaluate this statement.

A. False, because C does not allow multiple variables.

B. True, but only when the values are of different types.

C. True, because separate variables for many related values are impractical to manage.

D. False, because separate variables are always easier to manage.

Correct Answer: C

Explanation: The need for collections arises directly from the impracticality of using many individual variables.

Q4 | Definition | Concept of Collection

Which of the following best describes a "characteristic" of a collection?

A. It cannot be used inside loops.

B. It must always be declared as a constant.

C. It can only ever hold one element.

D. It groups multiple elements that can be referenced through a single identifier.

Correct Answer: D

Explanation: A defining characteristic of any collection is grouping multiple elements under one name.

Q5 | Application | Concept of Collection

Which of the following is a genuine advantage of using a collection over independent variables?

A. Guaranteed reduction of program size to zero.

B. Easier bulk processing, such as looping through all elements for a calculation.

C. Removal of the need for indexing.

D. Automatic conversion between data types.

Correct Answer: B

Explanation: Collections allow operations like loops to process many values together, which independent variables cannot offer as conveniently.

Q6 | Scenario | Concept of Collection

A program must store the ages of 100 people and later compute their average. Which approach best reflects the "need for collections"?

A. Using a single variable and overwriting it 100 times.

B. Using a single collection to store all 100 ages instead of 100 separate variables.

C. Avoiding storage of the ages altogether.

D. Declaring 100 separate int variables.

Correct Answer: B

Explanation: This scenario is precisely why collections are needed - to manage many related values efficiently.

Q7 | Comparison | Concept of Collection

How does a collection differ fundamentally from a single scalar variable?

A. There is no real difference between them.

B. A scalar variable can store more values than a collection.

C. A collection can never be looped over, unlike a scalar variable.

D. A collection stores and organizes multiple values under one identifier, whereas a scalar variable stores only a single value.

Correct Answer: D

Explanation: The essential difference is that a collection groups multiple values, while a scalar variable holds just one.

Q8 | Tricky | Concept of Collection

A student says, "Since a single variable can store a value, there is no need for collections in any program." Why is this statement flawed?

A. It is correct because a single variable can store unlimited values.

B. It is correct because collections are never used in real programs.

C. It ignores that many real-world programs must handle numerous related values, which a single variable cannot manage efficiently.

D. It is flawed because single variables cannot store any value at all.

Correct Answer: C

Explanation: The claim overlooks the practical need to manage multiple related values, which is the entire rationale for collections.

Chapter 2: Types of Collections Supported by C

6 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q9 | Basic | Types of Collections Supported by C

Which built-in collection type does the C language primarily provide for storing multiple values of the same data type?

A. Linked list (as a native built-in type)

B. Set

C. Array

D. Dictionary

Correct Answer: C

Explanation: C's core, built-in collection construct is the array; structures like linked lists must be built manually using arrays or pointers.

Q10 | Conceptual | Types of Collections Supported by C

Which is a key characteristic of an array as a collection in C?

A. Its size can change automatically at runtime.

B. All its elements are of the same data type and are stored in contiguous memory locations.

C. Its elements cannot be accessed individually.

D. Its elements can be of completely different, unrelated data types.

Correct Answer: B

Explanation: Arrays store same-typed elements contiguously, which enables indexed access.

Q11 | Application | Types of Collections Supported by C

Which situation is best suited to using an array as a collection?

A. Storing a single employee's ID number.

B. Storing the marks of 50 students to compute their average.

C. Storing one Boolean status flag.

D. Storing a single character constant.

Correct Answer: B

Explanation: Arrays suit situations with many same-type related values, such as 50 students' marks.

Q12 | Comparison | Types of Collections Supported by C

Compared to simple (scalar) variables, what advantage does an array collection provide?

A. It allows storage of only one value at a time.

B. It removes the need for any data type.

C. It allows many values to be stored and accessed together under a single name using indices.

D. It automatically sorts stored data.

Correct Answer: C

Explanation: Arrays let multiple related values be grouped under one identifier and accessed via index, unlike scalar variables.

Q13 | Statement-based | Types of Collections Supported by C

Statement: "C provides built-in dynamic collection types such as lists and dictionaries, similar to Python." Evaluate this statement.

A. False - C's main built-in collection structure is the array, not dynamic lists or dictionaries.

B. True - C has native list and dictionary types.

C. False - C has no way to group data at all.

D. True, but only in older C standards.

Correct Answer: A

Explanation: Unlike Python, C does not provide built-in dynamic collections; arrays are its primary native collection type.

Q14 | Tricky | Types of Collections Supported by C

A beginner claims that arrays in C automatically increase in size whenever new elements are added. Is this claim correct?

A. Yes, but only for character arrays.

B. No - arrays can never store more than one element.

C. Yes - arrays resize themselves as needed.

D. No - an array's size is fixed at the time of declaration and does not grow automatically.

Correct Answer: D

Explanation: Array size in C is fixed once declared; it does not grow dynamically like some higher-level language structures.

Chapter 3: Array Collection and its Features

8 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q15 | Definition | Array Collection and its Features

An array in C can be defined as:

A. A single variable that can hold any data type at once.

B. A keyword used to declare pointers.

C. A collection of elements of the same data type stored in contiguous memory locations under a single name.

D. A function that returns several values.

Correct Answer: C

Explanation: This is the standard definition of an array: same-type elements, contiguous storage, one name.

Q16 | Feature | Array Collection and its Features

Which of the following is a core feature of arrays in C?

A. Elements are accessed using an index number.

B. Elements are stored in random, non-contiguous memory.

C. Elements are accessed only through function calls.

D. Elements cannot be modified after declaration.

Correct Answer: A

Explanation: Indexed access is a defining feature of arrays, allowing direct access to any element.

Q17 | Advantage | Array Collection and its Features

Which of these is a genuine advantage of using arrays?

A. Arrays remove the need for specifying a data type.

B. Arrays automatically resize themselves during program execution.

C. Arrays can store unlimited elements without declaring a size.

D. Related data can be stored and processed together using loops, reducing code duplication.

Correct Answer: D

Explanation: Arrays enable compact, loop-based processing of related data, avoiding repetitive code for each variable.

Q18 | Limitation | Array Collection and its Features

Which of the following is a genuine limitation of arrays in C?

A. Their size is fixed at declaration and cannot be changed during execution.

B. They cannot be indexed.

C. They cannot store integer values.

D. They must always contain exactly one element.

Correct Answer: A

Explanation: A key limitation of C arrays is their static size, fixed once declared.

Q19 | Application | Array Collection and its Features

Which of the following is a typical real-world application of arrays?

A. Storing one constant value like pi.

B. Storing a single character for a grade.

C. Storing daily temperature readings for a month to calculate the monthly average.

D. Storing a single yes/no flag for login status.

Correct Answer: C

Explanation: Arrays are ideal for storing a series of related, same-type readings such as daily temperatures.

Q20 | Comparison | Array Collection and its Features

How do arrays differ from simple variables in terms of data organization?

A. Arrays organize multiple same-type values under one name; simple variables hold only a single value.

B. There is no organizational difference between them.

C. Arrays can hold only one value; simple variables hold many.

D. Arrays cannot be organized at all.

Correct Answer: A

Explanation: This distinction is the essential feature separating arrays (collections) from scalar (simple) variables.

Q21 | Assertion-Reason | Array Collection and its Features

Assertion (A): Arrays are useful for storing large numbers of related values. Reason (R): Arrays allow indexed access to individual elements. Choose the correct option.

A. A is false, but R is true.

B. Both A and R are false.

C. A is true, but R is false.

D. Both A and R are true, and R correctly explains A.

Correct Answer: D

Explanation: Indexed access (R) is precisely what makes arrays practical for storing and retrieving large sets of related values (A).

Q22 | Tricky | Array Collection and its Features

A programmer stores 10 integers using an array but later needs to store 15. What must be done, given the fixed-size limitation of arrays?

A. The program will resize the array at runtime without any changes.

B. The array will overwrite unrelated memory to make space.

C. Nothing - the array will automatically expand to fit 15 integers.

D. Declare a new, larger array (or redeclare with a bigger size), since the original array's size cannot grow.

Correct Answer: D

Explanation: Because array size is fixed at declaration, accommodating more elements requires declaring a new array with sufficient size.

Chapter 4: Concept of Indexing 8 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q23 | Definition | Concept of Indexing

"Indexing" in the context of arrays refers to:

A. The technique of converting an array into a string.

B. The method of identifying and accessing a specific element in an array using its position number.

C. A method used only for sorting arrays.

D. The process of compiling a C program.

Correct Answer: B

Explanation: Indexing is the mechanism that lets a specific array element be located and accessed by its position.

Q24 | Basic | Concept of Indexing

In C, array indexing begins from which value?

A. 1

B. 0

C. 2

D. -1

Correct Answer: B

Explanation: C arrays use zero-based indexing, so the first element is at index 0.

Q25 | Conceptual | Concept of Indexing

If an array has 10 elements, what is the valid index range?

A. 0 to 10

B. 1 to 9

C. 1 to 10

D. 0 to 9

Correct Answer: D

Explanation: With 10 elements and zero-based indexing, valid indices run from 0 through 9 (10 total positions).

Q26 | Positioning | Concept of Indexing

How does indexing help determine the position of an element in an array?

A. Indexing has no relation to element position.

B. Indexing only tells the total size of the array.

C. Indexing determines the data type of the array.

D. Each index value directly corresponds to the offset of an element from the start of the array.

Correct Answer: D

Explanation: The index value is essentially the position offset from the beginning of the array in memory.

Q27 | Access Mechanism | Concept of Indexing

Which of the following correctly accesses the 3rd element of an array named marks?

A. marks[2]

B. marks{2}

C. marks(3)

D. marks[3]

Correct Answer: A

Explanation: Since indexing starts at 0, the 3rd element is accessed using index 2, i.e., marks[2].

Q28 | Statement-based | Concept of Indexing

Statement: "Array indexing in C always starts from 1, similar to everyday counting." Evaluate this statement.

A. False - indexing starts from -1.

B. True - indexing always starts from 1.

C. True, but only for character arrays.

D. False - C array indexing always starts from 0, not 1.

Correct Answer: D

Explanation: This is a common misconception; C strictly uses zero-based indexing for arrays.

Q29 | Scenario | Concept of Indexing

An array int a[5]; is declared. Which index would cause an out-of-bounds access?

A. a[4]

B. a[2]

C. a[0]

D. a[5]

Correct Answer: D

Explanation: Valid indices for a 5-element array are 0 to 4; a[5] refers to a non-existent 6th element, causing an out-of-bounds access.

Q30 | Tricky | Concept of Indexing

Why is understanding indexing critical when working with arrays in C?

A. Because indexing determines the color of output text.

B. Because indexing changes the array's data type automatically.

C. Because C does not automatically check index bounds, incorrect indexing can access invalid memory.

D. Because indexing is optional and arrays can be accessed without it.

Correct Answer: C

Explanation: Since C performs no automatic bounds checking, understanding correct index usage is essential to avoid undefined behavior.

Chapter 5: Index Variable 6 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q31 | Definition | Index Variable

An "index variable" is best defined as:

A. A variable that stores the total size of an array.

B. A special constant defined by the compiler.

C. A variable used to hold the index value that specifies which array element is being accessed.

D. A variable that can only hold floating-point numbers.

Correct Answer: C

Explanation: An index variable stores the numeric position used to reference a specific array element.

Q32 | Purpose | Index Variable

What is the main purpose of using an index variable, such as i, in a loop that processes an array?

A. To store the sum of all array elements permanently.

B. To systematically refer to each array element in turn during iteration.

C. To declare the array's size dynamically.

D. To change the array's data type during the loop.

Correct Answer: B

Explanation: Index variables like i are used in loops to move through and access each element of the array sequentially.

Q33 | Characteristics | Index Variable

Which characteristic must an index variable in C generally have?

A. It must always be a character constant.

B. It should hold an integer value corresponding to a valid array position.

C. It must always be declared as a float.

D. It must be declared outside the program.

Correct Answer: B

Explanation: Since array indices are whole numbers representing positions, index variables are typically integers.

Q34 | Usage | Index Variable

In the code for(i=0; i<n; i++) arr[i] = 0;, what role does i play?

A. It defines the data type of arr.

B. It stores the final value of the array.

C. It represents the memory address of the compiler.

D. It acts as the index variable that accesses each element of arr in sequence.

Correct Answer: D

Explanation: Here, i is incremented each loop cycle and used as the index to access arr[i] sequentially.

Q35 | Application | Index Variable

Why is using an index variable more efficient than manually writing arr[0], arr[1], arr[2], and so on individually?

A. It removes the need for the array to have a declared size.

B. It converts the array into a different data type.

C. It reduces the array's memory usage to zero.

D. It allows the same code (using a loop) to process arrays of any size without repeating statements.

Correct Answer: D

Explanation: Index variables combined with loops let one line of code handle elements of any array size, avoiding repetition.

Q36 | Tricky | Index Variable

If the index variable i in a loop mistakenly runs from 1 to n (instead of 0 to n-1) for an array of size n, what happens?

A. The program automatically corrects itself and works fine.

B. The array size increases to accommodate index n.

C. The first element (index 0) is skipped, and an out-of-bounds access occurs at index n.

D. Nothing happens; all elements are processed correctly.

Correct Answer: C

Explanation: Since valid indices run from 0 to n-1, looping from 1 to n skips the first element and accesses one index beyond the array's bounds.

Chapter 6: Index Type

6 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q37 | Definition | Index Type

In C, array indices must be of which fundamental type?

A. Character string type

B. Boolean type only

C. Floating-point type

D. Integer type

Correct Answer: D

Explanation: Array indices in C must be integers (or integer-compatible types); floating-point or string values cannot be used directly as indices.

Q38 | Rule | Index Type

Which of the following is a valid index value for accessing an array element in C?

A. Any arbitrary string.

B. A Boolean value like true.

C. A non-negative integer within the array's declared size.

D. A negative floating-point number.

Correct Answer: C

Explanation: Valid indices are non-negative whole numbers that lie within the array's bounds.

Q39 | Restriction | Index Type

What restriction applies to index values used to access array elements in C?

A. They can be any real number, positive or negative.

B. They must always be exactly equal to the array size.

C. There is no restriction on index values in C.

D. They must be within the range 0 to (size - 1) for defined behavior.

Correct Answer: D

Explanation: Using an index outside 0 to size-1 leads to undefined behavior since C does not check bounds automatically.

Q40 | Statement-based | Index Type

Statement: "An array index can be a floating-point expression like arr[2.5] in standard C." Evaluate.

A. False - array indices must evaluate to integer values, not floating-point values.

B. False - indices must always be negative.

C. True, but only for two-dimensional arrays.

D. True - floating-point indices are always valid.

Correct Answer: A

Explanation: C requires indices to be integral; a floating-point expression like 2.5 is not a valid direct index.

Q41 | Application | Index Type

Which expression below is a valid way to index an array using a computed integer index?

A. arr[true && false] treated as a non-integer

B. arr["1"] using a string literal

C. arr[i + 1.5] where the result is fractional

D. arr[i + 1] where i is an integer variable

Correct Answer: D

Explanation: arr[i + 1] evaluates to an integer, making it a valid array index expression.

Q42 | **Tricky** | **Index Type**

A programmer writes arr[3.0] to access an array element. What is the likely outcome in C?

A. The index becomes invalid and always crashes the program.

B. The program refuses to compile under any circumstances.

C. The compiler converts 3.0 to an integer index (3), but relying on this is poor and non-portable practice.

D. The array automatically becomes a floating-point array.

Correct Answer: C

Explanation: While C may implicitly convert a floating literal to an integer in some contexts, correct practice always uses integer index expressions.

Chapter 7: Positional Value of a Member of Array Collection

6 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q43 | Definition | Positional Value of a Member of Array Collection

The "positional value" of an array element refers to:

A. The data type used for the array.

B. The total number of elements in the array.

C. The specific location/index of that element within the array's sequence.

D. The literal value stored at index 0 only.

Correct Answer: C

Explanation: Positional value describes where an element sits within the ordered sequence of the array, identified by its index.

Q44 | Addressing | Positional Value of a Member of Array Collection

How is the positional concept in arrays related to memory addressing?

A. Each position (index) corresponds to a specific offset from the base address of the array in memory.

B. Every element shares exactly the same memory address.

C. Positional concepts have no connection to memory addressing.

D. Position determines only the array's name, not its address.

Correct Answer: A

Explanation: Since arrays are stored contiguously, each index maps to a specific memory offset from the array's base address.

Q45 | Relationship | Positional Value of a Member of Array Collection

What is the relationship between an array's index and an element's position?

A. The index always equals the position plus 100.

B. The position is only meaningful for the last element.

C. The index directly represents the element's position, starting from 0 for the first element.

D. The index and position are unrelated.

Correct Answer: C

Explanation: In C, the index is the position identifier itself, beginning at 0.

Q46 | Access | Positional Value of a Member of Array Collection

To access the element at the 5th position of an array marks (using standard C indexing), which expression is correct?

A. marks[6]

B. marks[4]

C. marks(5)

D. marks[5]

Correct Answer: B

Explanation: The 5th positional element corresponds to index 4, since indexing starts at 0.

Q47 | Conceptual | Positional Value of a Member of Array Collection

Why does knowing an element's positional value matter when processing arrays?

A. It determines the array's total memory size automatically.

B. It allows precise access, modification, and reference to that specific element during program execution.

C. It only matters for arrays with a single element.

D. It has no practical use in programming.

Correct Answer: B

Explanation: Understanding positional value (index) is essential for correctly accessing or updating specific array elements.

Q48 | Tricky | Positional Value of a Member of Array Collection

In an array int a[6] = {10,20,30,40,50,60};, what is the positional value (index) of the element with value 40?

A. 4

B. 40

C. 2

D. 3

Correct Answer: D

Explanation: Counting from index 0: 10(0), 20(1), 30(2), 40(3) - so 40 is at index 3.

Chapter 8: Concept of Dimension and Size of an Array 8 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q49 | Definition | Concept of Dimension and Size of an Array

The "dimension" of an array refers to:

A. The name given to the array.

B. The data type of the array elements.

C. The number of indices required to access an individual element (e.g., one-dimensional, two-dimensional).

D. The total number of bytes the array occupies.

Correct Answer: C

Explanation: Dimension describes how many indices are needed to specify an element - one index for 1-D, two for 2-D, and so on.

Q50 | Definition | Concept of Dimension and Size of an Array

A one-dimensional array is best described as:

A. A collection that stores only characters.

B. A linear collection of elements accessed using a single index.

C. A collection with no fixed size.

D. A collection accessed using two indices.

Correct Answer: B

Explanation: A one-dimensional array is a simple linear list of elements, each accessed via one index value.

Q51 | Concept | Concept of Dimension and Size of an Array

What does the "size" of an array represent?

A. The total number of elements the array can hold.

B. The name of the array variable.

C. The data type used to declare the array.

D. The number of dimensions the array has.

Correct Answer: A

Explanation: Size refers to the count of elements an array can store, as specified at declaration.

Q52 | Storage | Concept of Dimension and Size of an Array

How is the total memory occupied by an array determined?

A. By adding the array's name length to its size.

B. By dividing the size by the number of dimensions.

C. By multiplying the number of elements (size) by the memory size of each element's data type.

D. It is always exactly 100 bytes regardless of type.

Correct Answer: C

Explanation: Total memory = size x size of each element's data type (e.g., 10 ints x 4 bytes = 40 bytes).

Q53 | Capacity | Concept of Dimension and Size of an Array

If int arr[20]; is declared, what is the array's capacity?

A. 20 bytes regardless of type

B. 19 integer elements

C. 20 integer elements

D. Unlimited elements

Correct Answer: C

Explanation: The declared size, 20, directly gives the array's capacity - it can hold 20 integers.

Q54 | Application | Concept of Dimension and Size of an Array

Which scenario best demonstrates the correct use of "size" in an array declaration?

A. Declaring int marks[30]; to store the marks of exactly 30 students.

B. Declaring int marks; to store 30 students' marks.

C. Declaring int marks[]; without ever using it with an initializer.

D. Declaring marks[30]; without specifying a data type.

Correct Answer: A

Explanation: The size, 30, exactly matches the number of elements to be stored, which is the correct use of array sizing.

Q55 | Statement-based | Concept of Dimension and Size of an Array

Statement: "The size of a standard fixed array in C must be specified as a non-negative integer constant expression at declaration." Evaluate.

A. False - sizes must always be floating-point.

B. True, but only for two-dimensional arrays.

C. False - sizes can be any negative number.

D. True - the size must be a non-negative integer constant expression.

Correct Answer: D

Explanation: Standard C array declarations require the size to be a non-negative integer constant expression.

Q56 | Tricky | Concept of Dimension and Size of an Array

An array is declared as float temp[15];. What is its "dimension" and "size" respectively?

A. Dimension: fifteen-dimensional; Size: 1 element

B. Dimension: two-dimensional; Size: 15 elements

C. Dimension: one-dimensional; Size: 1 element

D. Dimension: one-dimensional; Size: 15 elements

Correct Answer: D

Explanation: temp is a single-indexed (one-dimensional) array capable of holding 15 float elements.

Chapter 9: C Syntax for Declaration of Array

12 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q57 | Syntax | C Syntax for Declaration of Array

Which of the following is the correct general syntax for declaring a one-dimensional array in C?

A. data_type array_name[size];

B. data_type[size] array_name;

C. size array_name[data_type];

D. array data_type[size] name;

Correct Answer: A

Explanation: The standard syntax places the data type first, followed by the array name and the size in square brackets.

Q58 | Basic | C Syntax for Declaration of Array

Which of the following correctly declares an integer array of 10 elements?

A. int arr[10];

B. int arr(10);

C. array int arr[10];

D. integer arr[10];

Correct Answer: A

Explanation: int arr[10]; correctly specifies the data type, array name, and size in square brackets.

Q59 | Declaration Rule | C Syntax for Declaration of Array

Which of the following array declarations is INVALID in standard C?

A. float arr[20];

B. int arr[-5];

C. char arr[100];

D. int arr[5];

Correct Answer: B

Explanation: Array sizes must be non-negative; int arr[-5]; is invalid because negative sizes are not allowed.

Q60 | Naming Convention | C Syntax for Declaration of Array

Which of the following is a valid array name in C?

A. 2ndArray

B. studentMarks

C. student-marks

D. int

Correct Answer: B

Explanation: Identifiers in C must start with a letter or underscore and cannot use hyphens or reserved keywords like int.

Q61 | Valid Declaration | C Syntax for Declaration of Array

Which declaration correctly creates a character array capable of storing 20 characters?

A. char name[20];

B. character name[20];

C. char name(20);

D. char[20] name;

Correct Answer: A

Explanation: char name[20]; follows correct C syntax: data type, name, then size in brackets.

Q62 | Invalid Declaration | C Syntax for Declaration of Array

Which of the following is an invalid array declaration in C?

A. double arr[10];

B. int [10] arr;

C. int arr[10], brr[5];

D. int arr[10];

Correct Answer: B

Explanation: The size in square brackets must appear after the array name, not before it, making int [10] arr; invalid.

Q63 | Code-snippet | C Syntax for Declaration of Array

What does the declaration double prices[50]; create?

A. An array named prices capable of holding 50 double (floating-point) values.

B. An array of 50 integers named prices.

C. A function named prices that returns 50 doubles.

D. A single double variable named prices.

Correct Answer: A

Explanation: This declares an array of type double with 50 elements, named prices.

Q64 | Fill-in-the-Blank | C Syntax for Declaration of Array

Fill in the blank: To declare an array of 100 floating-point numbers named data, you write: _____ data[100];

A. char

B. int

C. void

D. float

Correct Answer: D

Explanation: Since the array stores floating-point numbers, float is the correct data type keyword to complete the declaration.

Q65 | Statement-based | C Syntax for Declaration of Array

Statement: "In C, the size specified in a basic array declaration must always be a variable, never a constant." Evaluate.

A. True, but only in function parameters.

B. False - the size in a basic array declaration is typically a constant expression, not necessarily a variable.

C. True - sizes must always be variables.

D. False - sizes cannot be numbers at all.

Correct Answer: B

Explanation: Standard fixed-size array declarations in C require the size to be a constant expression, not a variable (in classic C).

Q66 | Scenario | C Syntax for Declaration of Array

A programmer wants to declare an array to store the ages of 25 employees. Which declaration is most appropriate?

A. float age(25);

B. int age;

C. int age[age];

D. int age[25];

Correct Answer: D

Explanation: Ages are whole numbers, so int age[25]; correctly declares an integer array with 25 elements.

Q67 | Previous-Year Pattern | C Syntax for Declaration of Array

Which of the following correctly declares an array along with initialization in the same statement?

A. int arr{5} = [1];

B. int arr[5] = {1};

C. int arr[5] == {1};

D. int arr[5]; = {1};

Correct Answer: B

Explanation: int arr[5] = {1}; is valid syntax - it declares an array of size 5 and initializes the first element to 1 (the rest default to 0).

Q68 | Tricky | C Syntax for Declaration of Array

What is wrong with the declaration int arr[5.5];?

A. The array name is invalid.

B. The array size must be an integer value, but 5.5 is a floating-point number, making this invalid.

C. Nothing is wrong; this is perfectly valid C syntax.

D. The data type int cannot be used with arrays.

Correct Answer: B

Explanation: Array sizes must be integer constant expressions; using 5.5 (a float) as a size is a syntax error in standard C.

Chapter 10: Name of the Array and Its Type 6 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q69 | Definition | Name of the Array and Its Type

The "name" of an array serves which primary purpose?

A. It determines the number of dimensions automatically.

B. It specifies only the size of the array.

C. It has no functional purpose in C.

D. It acts as the identifier used to refer to the entire collection of elements.

Correct Answer: D

Explanation: The array name is the identifier through which the whole collection (and its elements via indexing) is accessed.

Q70 | Data Type | Name of the Array and Its Type

In the declaration int scores[10];, what does int specify?

A. The name of the array.

B. The number of dimensions of the array.

C. The data type of each element stored in the array.

D. The total size of the array in bytes only.

Correct Answer: C

Explanation: int specifies that every element in the scores array is an integer.

Q71 | Type Specification | Name of the Array and Its Type

Why must all elements of a standard C array share the same declared data type?

A. Because the array's type determines uniform memory allocation and consistent element access.

B. Because the array name would otherwise become invalid.

C. Because C requires every variable in a program to share one global type.

D. Because arrays cannot store numeric values otherwise.

Correct Answer: A

Explanation: A single data type ensures each element occupies the same amount of memory, enabling predictable indexed access.

Q72 | Declaration Analysis | Name of the Array and Its Type

Given the declaration char grade[5];, what can be concluded?

A. grade is a single character variable.

B. grade is an array of 5 integers.

C. grade is an array of 5 characters.

D. grade is a function returning characters.

Correct Answer: C

Explanation: The syntax indicates an array named grade, of type char, with 5 elements.

Q73 | Application | Name of the Array and Its Type

Which factor determines how many bytes each element of an array occupies in memory?

A. The programming environment's operating system only.

B. The data type specified when the array is declared.

C. The number of elements in the array.

D. The name chosen for the array.

Correct Answer: B

Explanation: The declared data type (e.g., int, float, char) determines the memory size of each individual element.

Q74 | Tricky | Name of the Array and Its Type

Two arrays are declared as int a[10]; and float b[10];. Why do they occupy different total memory even though both have 10 elements?

A. Because int and float have different sizes per element, so total memory (size x element size) differs.

B. Because one array has more dimensions than the other.

C. Because their names are different lengths.

D. Because arrays with the same size always use the same memory regardless of type.

Correct Answer: A

Explanation: Memory usage depends on element size (dictated by data type) multiplied by number of elements; int and float differ in per-element size.

Chapter 11: Referring Individual Elements

8 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q75 | Access | Referring Individual Elements

How is an individual element of an array referred to in C?

A. By using the array name followed by the element's index in square brackets, e.g., arr[i].

B. By using the array name alone, without any index.

C. By using a separate function call for every element.

D. By using the array name followed by parentheses, e.g., arr(i).

Correct Answer: A

Explanation: Individual elements are accessed via index notation: array_name[index].

Q76 | Read Operation | Referring Individual Elements

Which statement correctly reads (retrieves) the value of the 4th element of array data into variable x?

A. x = data[4];

B. x = data(3);

C. data[3] = x;

D. x = data[3];

Correct Answer: D

Explanation: Since indexing starts at 0, the 4th element is data[3]; assigning it to x reads its value.

Q77 | Write Operation | Referring Individual Elements

Which statement correctly writes (assigns) the value 50 to the first element of array marks?

A. 50 = marks[0];

B. marks[0] = 50;

C. marks[0] == 50;

D. marks[1] = 50;

Correct Answer: B

Explanation: The first element is at index 0, and assignment uses =, so marks[0] = 50; is correct.

Q78 | Index Notation | Referring Individual Elements

In the expression arr[i+2], what does i+2 represent?

A. A comment ignored by the compiler.

B. The computed index used to access a specific element of arr.

C. A new array being declared.

D. The total size of arr.

Correct Answer: B

Explanation: Any valid integer expression, like i+2, can be used inside square brackets to compute the index.

Q79 | Code-snippet | Referring Individual Elements

What does the statement printf("%d", list[2]); output, given int list[5] = {5,10,15,20,25};?

A. 5

B. The entire array

C. 10

D. 15

Correct Answer: D

Explanation: list[2] refers to the third element (index 2), which holds the value 15.

Q80 | Scenario | Referring Individual Elements

A program needs to double the value of the 6th element of an array called nums. Which statement achieves this?

A. nums[6] = nums[6] * 2;

B. nums(5) = nums(5) * 2;

C. nums[5] = nums[5] * 2;

D. nums[5] * 2 = nums[5];

Correct Answer: C

Explanation: The 6th element is at index 5; multiplying it by 2 and reassigning doubles its value correctly.

Q81 | Statement-based | Referring Individual Elements

Statement: "Individual array elements in C can be both read from and written to using index notation." Evaluate.

A. True, but only for the first element.

B. False - index notation only allows writing, never reading.

C. True - index notation supports both retrieving and assigning values to array elements.

D. False - index notation only allows reading, never writing.

Correct Answer: C

Explanation: arr[i] can appear on the right side of an assignment (read) or the left side (write), supporting both operations.

Q82 | Tricky | Referring Individual Elements

What is the output of: int a[4]={1,2,3,4}; a[1]=a[1]+a[2]; printf("%d",a[1]);

A. 2

B. 7

C. 3

D. 5

Correct Answer: D

Explanation: a[1] (2) is added to a[2] (3), giving 5, which is then stored back into a[1] and printed.

Chapter 12: Entering Data into an Array 8 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q83 | Input Concept | Entering Data into an Array

Which of the following best describes "entering data" into an array?

A. Declaring an array without ever assigning it any value.

B. Assigning values to array elements either through initialization or user/program input.

C. Converting an array into a single variable.

D. Deleting all values from an array.

Correct Answer: B

Explanation: Entering data refers to the process of placing values into array elements, whether by initialization or input statements.

Q84 | Code-snippet | Entering Data into an Array

Which code snippet correctly reads 5 integers from the user into an array arr of size 5?

A. for(i=1;i<=5;i++) scanf("%d",arr[i]);

B. for(i=0;i<5;i++) scanf("%d",&arr[i]);

C. scanf("%d",&arr);

D. for(i=0;i<5;i++) scanf("%d",arr[i]);

Correct Answer: B

Explanation: The address-of operator (&) must be used with scanf, and the correct loop range (0 to 4) safely fills all 5 elements.

Q85 | Assignment | Entering Data into an Array

Which statement directly assigns the value 25 to the third element of an integer array score?

A. score[3] = 25;

B. score = 25[2];

C. score[2] = 25;

D. 25 = score[2];

Correct Answer: C

Explanation: The third element is at index 2; direct assignment uses score[2] = 25;.

Q86 | Storage Behavior | Entering Data into an Array

What happens if a program only assigns values to some elements of a partially initialized static/global array?

A. The program will refuse to compile.

B. The entire array automatically fills with the value 100.

C. The unassigned elements are automatically set to zero.

D. The array's size automatically shrinks to match the assigned elements.

Correct Answer: C

Explanation: For static/global arrays, elements not explicitly given a value in a partial initializer default to zero.

Q87 | Scenario | Entering Data into an Array

A program must collect 10 temperature readings entered by the user and store them for later processing. What is the appropriate approach?

A. Use a single variable and overwrite it after each reading.

B. Ignore user input and hard-code 10 values.

C. Store all readings in the array's name variable directly.

D. Use a loop with scanf to store each reading into successive elements of a 10-element array.

Correct Answer: D

Explanation: A loop combined with array indexing allows all 10 readings to be captured and retained for later use.

Q88 | Statement-based | Entering Data into an Array

Statement: "Values can be entered into an array either at compile-time through initialization or at run-time through input statements like scanf." Evaluate.

A. True - both compile-time initialization and run-time input are valid ways of entering data into an array.

B. False - arrays can only be filled at run-time.

C. False - arrays can only be filled at compile-time.

D. True, but only using printf.

Correct Answer: A

Explanation: Both initializer lists and functions like scanf are legitimate methods for populating array elements.

Q89 | Application | Entering Data into an Array

Why is a loop typically used when entering multiple values into an array via user input?

A. Because loops automatically sort input values.

B. Because loops convert the array into a different data type.

C. Because it allows repeated input operations to fill each element without writing a separate scanf statement for every index.

D. Because loops are required by the C compiler to declare arrays.

Correct Answer: C

Explanation: Loops let one scanf statement be reused across all array indices, avoiding repetitive code.

Q90 | Tricky | Entering Data into an Array

In the loop for(i=0;i<=5;i++) scanf("%d",&arr[i]); for an array declared as int arr[5];, what is the problem?

A. The problem is that scanf cannot be used with arrays.

B. The problem is that & should not be used with arr[i].

C. The loop condition i<=5 causes an out-of-bounds write at index 5, since valid indices are only 0 to 4.

D. There is no problem; the loop works perfectly for a 5-element array.

Correct Answer: C

Explanation: Since valid indices for arr[5] range from 0 to 4, the condition i<=5 causes the loop to write one element beyond the array's bounds.

Chapter 13: Reading Data from an Array 6 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q91 | Retrieval Concept | Reading Data from an Array

"Reading data from an array" refers to:

A. Deleting elements from the array.

B. Assigning brand-new values into the array unconditionally.

C. Declaring the array for the first time.

D. Retrieving and using the values already stored at specific array indices.

Correct Answer: D

Explanation: Reading refers to accessing/retrieving existing values from array elements, as opposed to writing new ones.

Q92 | Access Method | Reading Data from an Array

Which statement correctly reads and prints the value stored in the 2nd element of array data?

A. printf("%d", data[2]);

B. scanf("%d", data[1]);

C. printf("%d", data[1]);

D. printf("%d", &data[1]);

Correct Answer: C

Explanation: The 2nd element is at index 1; printf correctly reads and displays its value.

Q93 | Sequential Reading | Reading Data from an Array

Which loop correctly reads (prints) every element of an integer array arr of size n sequentially?

A. for(i=0;i<n;i++) printf("%d ", arr[i]);

B. for(i=n;i>0;i--) printf("%d ", arr[i]);

C. for(i=1;i<=n;i++) printf("%d ", arr[i]);

D. for(i=0;i<n;i++) printf("%d ", arr);

Correct Answer: A

Explanation: Looping i from 0 to n-1 and printing arr[i] correctly reads every valid element in sequence.

Q94 | Referencing | Reading Data from an Array

Which expression correctly references the last element of an array int arr[8]; using its known size?

A. arr[8]

B. arr[7]

C. arr[size]

D. arr[-1]

Correct Answer: B

Explanation: For an 8-element array, the last valid index is 7 (size - 1).

Q95 | Statement-based | Reading Data from an Array

Statement: "Reading a value from an array does not modify the original data stored in it." Evaluate.

A. True, but only for the first element of the array.

B. True - reading simply retrieves the stored value without altering it.

C. False - reading deletes the accessed element.

D. False - reading always changes the stored value to zero.

Correct Answer: B

Explanation: Reading (e.g., via printf or assignment to another variable) is a non-destructive operation on array data.

Q96 | **Tricky** | **Reading Data from an Array**

Given int arr[3] = {7,14,21}; and int x = arr[3];, what is the concern with this statement?

A. There is no concern; arr[3] correctly reads the value 21.

B. x will automatically and safely become 0.

C. arr[3] is an out-of-bounds access, since valid indices for a 3-element array are only 0 to 2.

D. This will always print 14 with no issue.

Correct Answer: C

Explanation: Since the array has only indices 0-2, arr[3] accesses memory outside the array, leading to undefined behavior.

Chapter 14: Array Initialization

10 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q97 | Concept | Array Initialization

"Array initialization" refers to:

A. Declaring an array without specifying its name.

B. Deleting the array after it is no longer needed.

C. Assigning initial values to array elements at the time the array is declared.

D. Changing the array's data type during execution.

Correct Answer: C

Explanation: Initialization is the process of providing starting values to array elements right when the array is declared.

Q98 | Method | Array Initialization

Which of the following correctly initializes an integer array of 5 elements with the values 1 through 5?

A. int arr[5] = {1,2,3,4,5};

B. int arr = {1,2,3,4,5}[5];

C. int arr[5] == {1,2,3,4,5};

D. int arr[5] = (1,2,3,4,5);

Correct Answer: A

Explanation: Curly braces with comma-separated values correctly initialize an array at declaration.

Q99 | Partial Initialization | Array Initialization

Given int arr[5] = {1,2}; what are the values of arr[2], arr[3], and arr[4]?

A. The array declaration becomes invalid.

B. All three remain uninitialized garbage values.

C. All three are automatically set to 1.

D. All three are automatically set to 0.

Correct Answer: D

Explanation: When an array is partially initialized, the C standard guarantees the remaining elements are set to zero.

Q100 | Complete Initialization | Array Initialization

Which of the following represents complete initialization of a 4-element array?

A. int arr[4] = {10, 20, 30, 40};

B. int arr[4] = {};

C. int arr[4] = {10};

D. int arr[4] = {10, 20};

Correct Answer: A

Explanation: All four positions are explicitly given values, making this a complete initialization.

Q101 | Default Values | Array Initialization

If an array is declared as a global/static array without explicit initialization, what are its default element values?

A. The array cannot be declared without initialization.

B. All elements default to 1.

C. All elements default to garbage/unpredictable values.

D. All elements default to 0.

Correct Answer: D

Explanation: Global and static arrays in C are automatically zero-initialized by default, unlike local (automatic) arrays.

Q102 | Statement-based | Array Initialization

Statement: "A local (non-static) array that is not explicitly initialized contains garbage values." Evaluate.

A. True - uninitialized local arrays typically hold indeterminate garbage values.

B. True, but only for character arrays.

C. False - local arrays always default to 1.

D. False - local arrays are always zero by default.

Correct Answer: A

Explanation: Unlike static/global arrays, local arrays without explicit initialization contain unpredictable garbage values.

Q103 | Syntax | Array Initialization

Which of these initializations is syntactically INVALID in C?

A. int arr[3] = {1,2,3};

B. int arr[3] = {1,2,3,4};

C. int arr[3] = {1,2};

D. int arr[] = {1,2,3};

Correct Answer: B

Explanation: Providing more initializer values (4) than the declared size (3) is a compile-time error in C.

Q104 | Scenario | Array Initialization

A programmer wants an array of 100 integers, all initialized to zero, without listing 100 zeros manually. What is the simplest correct approach?

A. int arr[100] = 0;

B. int arr[100] = {0};

C. int arr[100] = {}; (relying on empty braces in classic C)

D. Typing {0,0,0,...} manually 100 times

Correct Answer: B

Explanation: Writing {0} sets the first element to 0, and by the rule for partial initialization, all remaining elements are automatically zeroed too.

Q105 | Comparison | Array Initialization

How does partial initialization differ from complete initialization of an array?

A. Partial initialization is only allowed for character arrays.

B. Complete initialization always leaves some elements as garbage.

C. Partial initialization provides values for only some elements (remaining become 0), while complete initialization provides values for every element.

D. They are identical in every respect.

Correct Answer: C

Explanation: Partial initialization leaves the remaining elements zero-filled, whereas complete initialization explicitly sets every element.

Q106 | Tricky | Array Initialization

What is the value of arr[4] after the declaration int arr[5] = {1,2,3};?

A. 0

B. 3

C. Garbage value

D. Undefined and unpredictable

Correct Answer: A

Explanation: Since only 3 values are provided for a 5-element array, the remaining elements - including arr[4] - are automatically initialized to 0.

Chapter 15: List of Initializers 8 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q107 | Concept | List of Initializers

An "initializer list" in an array declaration refers to:

A. A list of all variable names used in the program.

B. A separate array used only for storing error codes.

C. A function that lists array sizes.

D. The comma-separated set of values enclosed in braces used to give initial values to array elements.

Correct Answer: D

Explanation: The initializer list is the { value1, value2, ... } syntax used to assign starting values to an array.

Q108 | Rules | List of Initializers

Which rule must be followed when providing an initializer list for an array with an explicit size?

A. The number of initializer values must not exceed the declared size of the array.

B. Initializer lists cannot contain the number 0.

C. The number of initializer values must always exceed the declared size.

D. Initializer values must always be in descending order.

Correct Answer: A

Explanation: C requires that the initializer list not have more elements than the declared array size, or a compile error occurs.

Q109 | Valid Initialization | List of Initializers

Which of the following is a valid initializer list for int arr[4];?

A [5, 10, 15, 20]

B {5, 10, 15, 20, 25}

C {5, 10, 15, 20}

D (5, 10, 15, 20)

Correct Answer: C

Explanation: Curly braces with exactly (or fewer than) 4 comma-separated values correctly initialize a 4-element array.

Q110 | Syntax | List of Initializers

What is the correct syntax for initializing a character array to hold the characters 'A', 'B', 'C'?

A char arr[3] == {'A','B','C'};

B char arr[3] = {'A', 'B', 'C'};

C char arr[3] = "A, B, C";

D char arr[3] = ('A', 'B', 'C');

Correct Answer: B

Explanation: Character literal values, enclosed in single quotes, are correctly placed inside a brace-enclosed initializer list.

Q111 | Examples | List of Initializers

Which declaration correctly uses an initializer list to set up an array without explicitly specifying its size?

A int arr[] = (2,4,6,8);

B int arr[4] = {};

C int arr = {2,4,6,8};

D int arr[] = {2, 4, 6, 8};

Correct Answer: D

Explanation: When the size is omitted, the compiler infers it from the number of elements in the initializer list - here, 4.

Q112 | Statement-based | List of Initializers

Statement: "If the initializer list has fewer elements than the declared array size, the remaining elements are set to zero." Evaluate.

A. True, but only for arrays of size greater than 100.

B. True - this is the standard rule for partial initialization in C.

C. False - remaining elements always stay as garbage values.

D. False - the program will fail to compile in this case.

Correct Answer: B

Explanation: This is a fundamental and guaranteed rule of array initialization in C.

Q113 | Application | List of Initializers

Why might a programmer prefer int arr[] = {1,2,3,4,5}; over int arr[5] = {1,2,3,4,5};?

A. To let the compiler automatically determine the size from the initializer list, reducing chances of size-mismatch errors.

B. Because omitting the size changes the array's data type.

C. Because omitting the size makes the array store unlimited elements.

D. Because the C compiler requires the size to always be omitted.

Correct Answer: A

Explanation: Letting the compiler count elements from the initializer list avoids manual counting errors and keeps the size consistent with the list.

Q114 | Tricky | List of Initializers

What is the size of the array declared as int arr[] = {3, 6, 9, 12, 15, 18};?

A. 7

B. 6

C. 5

D. Undefined, since no size was specified

Correct Answer: B

Explanation: The compiler counts the six comma-separated values in the initializer list and sets the array size to 6 accordingly.

Chapter 16: Size Option

6 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q115 | Explicit Size | Size Option

What does "explicit size specification" mean in an array declaration such as int arr[10];?

A. The programmer directly states the number of elements (10) the array will hold.

B. The compiler guesses the array size at random.

C. The size refers to the data type used.

D. The size is determined only at program termination.

Correct Answer: A

Explanation: Explicit size specification means the array's capacity is directly written in the declaration by the programmer.

Q116 | Implicit Size | Size Option

In the declaration int arr[] = {4, 8, 12};, how is the array's size determined?

A. Implicitly, based on the number of values (3) in the initializer list.

B. It is always fixed at 10 by default.

C. Explicitly, since 3 is written in the brackets.

D. It must be specified separately using a size() function.

Correct Answer: A

Explanation: Since no size is given in the brackets, the compiler implicitly counts the initializer list's elements (3) to determine the size.

Q117 | **Relationship** | **Size Option**

What happens when both an explicit size and a shorter initializer list are provided, e.g., int arr[6] = {1,2,3};?

A. The compiler throws an error whenever both are used together.

B. The array becomes exactly 3 elements, ignoring the declared size.

C. The initializer list size (3) overrides the explicit size (6), shrinking the array.

D. The explicit size (6) takes precedence, and any unspecified elements beyond the initializer list default to 0.

Correct Answer: D

Explanation: When an explicit size is larger than the initializer list, the explicit size is honored, and remaining elements default to zero.

Q118 | **Capacity Concept** | **Size Option**

Which of these declarations relies entirely on implicit size determination?

A. float temp[10];

B. float temp[5] = {36.5, 37.0, 98.6};

C. float temp[3] = {36.5, 37.0, 98.6};

D. float temp[] = {36.5, 37.0, 98.6};

Correct Answer: D

Explanation: Since no size is given inside the brackets, the array's capacity (3) is implicitly determined from the initializer list.

Q119 | **Statement-based** | **Size Option**

Statement: "When explicit size and initializer list length do not match (the list is longer), the program compiles successfully." Evaluate.

A. False - providing more initializer values than the declared size causes a compilation error.

B. True - extra values automatically increase the array size.

C. False - the program always crashes at runtime instead of at compile time.

D. True - extra values are simply ignored, and the program compiles fine.

Correct Answer: A

Explanation: An initializer list containing more elements than the explicitly declared size results in a compile-time error in standard C.

Q120 | **Tricky** | **Size Option**

How do int a[5] = {1,2,3}; and int b[] = {1,2,3}; differ?

A. a has a fixed size of 5 with the last two elements defaulting to 0, while b's size is implicitly determined as exactly 3.

B. They are functionally identical in every respect, including size.

C. b has size 5 as well, matching a.

D. a has size 3 and b has size 5.

Correct Answer: A

Explanation: a explicitly reserves 5 slots (with 2 defaulting to zero), while b's size is inferred purely from its 3 initializer values.

Chapter 17: Bounds Checking

8 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q121 | Concept | Bounds Checking

"Bounds checking" in the context of arrays refers to:

A. Ensuring the array is printed in ascending order.

B. Checking whether the array's data type matches its name.

C. Verifying that the program has no syntax errors.

D. Verifying that an index used to access an array falls within its valid, declared range.

Correct Answer: D

Explanation: Bounds checking is specifically about validating that indices stay within the array's valid range.

Q122 | Array Limits | Bounds Checking

For an array declared as int arr[8];, what are its valid index limits?

A. 1 to 7

B. 0 to 7

C. 1 to 8

D. 0 to 8

Correct Answer: B

Explanation: With 8 elements and zero-based indexing, valid indices range from 0 to 7 inclusive.

Q123 | Valid Indices | Bounds Checking

Which of the following indices is valid for the array int data[12];?

A. -1

B. 11

C. 12

D. 13

Correct Answer: B

Explanation: For a 12-element array, the highest valid index is 11 (size - 1).

Q124 | Out-of-Bound Conditions | Bounds Checking

Which condition describes an "out-of-bounds" access for an array int arr[6];?

A. Accessing arr[5].

B. Accessing arr[0].

C. Accessing arr[6] or any index beyond 0-5.

D. Accessing arr[3].

Correct Answer: C

Explanation: Valid indices for a 6-element array run from 0 to 5; index 6 (or higher) is outside these bounds.

Q125 | Consequences | Bounds Checking

What is a key consequence of C's lack of automatic array bounds checking?

A. Out-of-bounds access may silently corrupt adjacent memory instead of raising an immediate error.

B. Out-of-bounds access is always caught and safely ignored at runtime.

C. The compiler always halts compilation if bounds are exceeded.

D. Arrays automatically expand to accommodate any index used.

Correct Answer: A

Explanation: Since C does not check bounds at runtime, invalid indices can lead to undefined behavior, such as silently corrupting nearby memory.

Q126 | Statement-based | Bounds Checking

Statement: "C automatically prevents programs from accessing array indices outside the declared bounds." Evaluate.

A. False - C does not perform automatic bounds checking on array accesses.

B. False - C prevents this only for two-dimensional arrays.

C. True, but only for character arrays.

D. True - C always halts the program upon invalid index access.

Correct Answer: A

Explanation: Unlike some higher-level languages, C provides no built-in bounds checking, leaving this responsibility to the programmer.

Q127 | Scenario | Bounds Checking

A loop is written as for(i=0;i<=n;i++) processes arr[i]; for an array int arr[n];. What issue does this raise regarding bounds checking?

A. There is no issue; the loop correctly processes all n elements.

B. The issue is unrelated to bounds and is purely a syntax error.

C. The condition i<=n allows i to reach n, causing an out-of-bounds access since valid indices only go up to n-1.

D. The loop accesses too few elements, missing the last one.

Correct Answer: C

Explanation: Using i<=n instead of i<n causes the loop to access index n, one position beyond the array's valid range (0 to n-1).

Q128 | Tricky | Bounds Checking

Why is careful bounds checking especially critical for arrays declared inside functions using local (automatic) storage?

A. Because writing beyond bounds may overwrite other local variables or control data, leading to unpredictable or dangerous behavior.

B. Because bounds checking only matters for global arrays, not local ones.

C. Because local arrays cannot be indexed at all.

D. Because local arrays automatically resize when bounds are exceeded, so checking is unnecessary.

Correct Answer: A

Explanation: Local arrays are typically stored in a region of memory that may hold other important data, so out-of-bounds writes can corrupt that data unpredictably.

Chapter 18: Two-Dimensional Arrays 16 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q129 | Concept | Two-Dimensional Arrays

A two-dimensional array can best be described as:

A. A collection where each element is itself a function.

B. A collection of elements arranged in rows and columns, accessed using two indices.

C. A collection that can hold only character data.

D. A collection of elements accessed using only one index.

Correct Answer: B

Explanation: Two-dimensional arrays organize data in a row-and-column (matrix-like) structure, requiring two indices to access an element.

Q130 | Declaration | Two-Dimensional Arrays

Which of the following correctly declares a two-dimensional integer array with 3 rows and 4 columns?

A. int arr[3,4];

B. int arr[3+4];

C. int arr[3][4];

D. int arr(3)(4);

Correct Answer: C

Explanation: The correct syntax for a 2-D array specifies each dimension in its own set of square brackets: [rows][columns].

Q131 | Syntax | Two-Dimensional Arrays

Which syntax correctly accesses the element in the 2nd row and 3rd column of a 2-D array named grid?

A grid(1)(2)

B grid[1][2]

C grid[2][3]

D grid[1,2]

Correct Answer: B

Explanation: Since indexing starts at 0, the 2nd row is index 1 and the 3rd column is index 2, giving grid[1][2].

Q132 | Dimensions | Two-Dimensional Arrays

How many indices are required to access a single element of a two-dimensional array?

A One index only.

B No index is required.

C Two - one for the row, one for the column.

D Three indices.

Correct Answer: C

Explanation: A 2-D array requires two indices - row index and column index - to identify a single element.

Q133 | Initialization | Two-Dimensional Arrays

Which of the following correctly initializes a 2x2 integer array with the values 1,2,3,4?

A int arr[2][2] = {1,2,3,4,5};

B int arr[2][2] = {{1,2},{3,4}};

C int arr[2][2] = (1,2,3,4);

D int arr[2][2] = {1,2},{3,4};

Correct Answer: B

Explanation: Nested braces group each row's values, correctly initializing the 2x2 array as {{1,2},{3,4}}.

Q134 | Element Access | Two-Dimensional Arrays

Given int a[2][3] = {{1,2,3},{4,5,6}};, what is the value of a[1][2]?

A. 3

B. 5

C. 4

D. 6

Correct Answer: D

Explanation: Row index 1 corresponds to {4,5,6}, and column index 2 within that row gives the value 6.

Q135 | Application | Two-Dimensional Arrays

Which real-world scenario is best modeled using a two-dimensional array?

A. Storing one exam's pass/fail status.

B. Storing a single integer constant.

C. Storing a single student's name.

D. Storing a table of student marks across multiple subjects for multiple students.

Correct Answer: D

Explanation: A table of data (students x subjects) naturally maps to rows and columns, making a 2-D array ideal.

Q136 | Statement-based | Two-Dimensional Arrays

Statement: "A two-dimensional array in C is essentially stored as a single, contiguous block of memory arranged row by row." Evaluate.

A. True, but only for character-type 2-D arrays.

B. False - each row of a 2-D array is stored in a completely separate, unrelated memory block.

C. True - 2-D arrays are stored in contiguous memory, typically in row-major order.

D. False - 2-D arrays cannot be stored contiguously in C.

Correct Answer: C

Explanation: C stores two-dimensional arrays in row-major order within one contiguous memory block.

Q137 | Code-snippet | Two-Dimensional Arrays

What does the following nested loop print for int a[2][2] = {{5,10},{15,20}};? for(i=0;i<2;i++) for(j=0;j<2;j++) printf("%d ", a[i][j]);

A. 5 15 10 20

B. 5 10 15 20

C. 20 15 10 5

D. 5 10

Correct Answer: B

Explanation: The nested loop prints row by row: first row (5,10), then second row (15,20), giving "5 10 15 20".

Q138 | Scenario | Two-Dimensional Arrays

A program needs to store the results of a 4-team, 4-round tournament (each team's score per round). Which structure is most suitable?

A. A two-dimensional array with 4 rows (teams) and 4 columns (rounds).

B. A single scalar integer variable.

C. Four completely unrelated separate programs.

D. A single one-dimensional array of size 4.

Correct Answer: A

Explanation: Since data varies across two categories (teams and rounds), a 2-D array (rows x columns) is the natural structure.

Q139 | Comparison | Two-Dimensional Arrays

How does a two-dimensional array differ from a one-dimensional array in terms of access?

A. A 2-D array requires no indices at all.

B. A 1-D array requires two indices, while a 2-D array requires one.

C. A 2-D array requires two indices (row and column) to access an element, while a 1-D array requires only one.

D. There is no difference in the number of indices required.

Correct Answer: C

Explanation: The fundamental access difference is the number of indices: one for 1-D arrays, two for 2-D arrays.

Q140 | Fill-in-the-Blank | Two-Dimensional Arrays

Fill in the blank: In the declaration int mat[4][5];, the array has ____ rows and 5 columns.

A. 4

B. 5

C. 20

D. 9

Correct Answer: A

Explanation: The first bracketed number (4) specifies the number of rows in the two-dimensional array.

Q141 | Assertion-Reason | Two-Dimensional Arrays

Assertion (A): Two-dimensional arrays are useful for representing matrices. Reason (R): They organize data using two indices representing rows and columns. Choose the correct option.

A. Both A and R are false.

B. A is true, but R is false.

C. Both A and R are true, and R correctly explains A.

D. A is false, but R is true.

Correct Answer: C

Explanation: The row-and-column indexing (R) is exactly why two-dimensional arrays naturally represent matrix-like data (A).

Q142 | Previous-Year Pattern | Two-Dimensional Arrays

What is the total number of elements in the array int arr[5][6];?

A. 30

B. 36

C. 11

D. 25

Correct Answer: A

Explanation: Total elements = rows x columns = 5 x 6 = 30.

Q143 | Tricky | Two-Dimensional Arrays

What is wrong with the declaration int arr[3][]; intended to create a 3-row 2-D array with unspecified columns?

A. Both dimensions must always be omitted together.

B. Nothing is wrong; this is a fully valid declaration in standard C.

C. The row size, not the column size, must always be omitted.

D. In a multi-dimensional array declaration, only the first (leftmost) dimension may be omitted; the column size cannot be left unspecified without an initializer that determines it.

Correct Answer: D

Explanation: In C, all dimensions except the first must be explicitly specified (or determinable from an initializer) - leaving the column size blank here is invalid.

Q144 | Difficult | Two-Dimensional Arrays

For a 2-D array declared as int arr[3][4];, how is the element arr[2][3] typically located in terms of row-major storage relative to the array's start?

A. It is located at offset (3 x 3 + 2), since arrays are stored column by column in C.

B. It is always located exactly at the midpoint of the array regardless of size.

C. It is located at an offset equal to (2 x 4 + 3) element positions from the start, since the array is stored row by row.

D. Row-major storage does not apply to 2-D arrays in C.

Correct Answer: C

Explanation: In row-major order, the offset of arr[i][j] is (i x number_of_columns + j); here (2x4+3) = 11 positions from the start.

Chapter 19: Similarities Between Dimension and Nesting

6 Questions

OAVS TGT Computer Science — Arrays & Collections Syllabus

Q145 | Concept | Similarities Between Dimension and Nesting

How is the concept of a two-dimensional array related to "nesting"?

A. A two-dimensional array can be conceptually viewed as an array of one-dimensional arrays (an array nested within an array).

B. Nesting only applies to functions, never to arrays.

C. A two-dimensional array is simply a renamed one-dimensional array.

D. A two-dimensional array has no relationship to nesting at all.

Correct Answer: A

Explanation: Conceptually, each row of a 2-D array behaves like a separate one-dimensional array, making the whole structure a "nested" arrangement of arrays.

Q146 | Comparison | Similarities Between Dimension and Nesting

What is a key similarity between the initializer syntax of a two-dimensional array and the concept of nesting?

A. Both use nested (braces within braces) grouping - an outer set of braces containing inner sets for each row.

B. Both require exactly one pair of braces with no nesting at all.

C. Nesting is only used for loops, not for initializer syntax.

D. Neither uses braces in their syntax.

Correct Answer: A

Explanation: The initializer {{1,2},{3,4}} mirrors nesting, with an outer collection (rows) containing inner collections (columns).

Q147 | Conceptual | Similarities Between Dimension and Nesting

In what sense can a one-dimensional array be considered the "building block" of a two-dimensional array?

A. A 1-D array can only exist independently and never as part of a larger structure.

B. Each row of a 2-D array is itself essentially a one-dimensional array, and the 2-D array is a collection of such rows.

C. A 2-D array is built from functions, not from 1-D arrays.

D. A one-dimensional array has no structural relationship to a two-dimensional array.

Correct Answer: B

Explanation: This reflects the nested relationship: a 2-D array is essentially a "collection of collections," with each row being a 1-D array.

Q148 | Statement-based | Similarities Between Dimension and Nesting

Statement: "The relationship between array dimensions and nesting shows that higher-dimensional arrays are structurally built from lower-dimensional ones." Evaluate.

A. False - dimensions and nesting are entirely unrelated concepts.

B. True - a two-dimensional array is structurally a nested arrangement of one-dimensional arrays.

C. False - higher-dimensional arrays have nothing to do with lower-dimensional ones.

D. True, but only for arrays of characters.

Correct Answer: B

Explanation: This nested relationship (arrays of arrays) is central to understanding multi-dimensional array concepts.

Q149 | Comparative | Similarities Between Dimension and Nesting

Which comparison best captures the similarity between "dimension" and "nesting" in array concepts?

A. Increasing dimension in arrays corresponds to increasing levels of nested grouping of elements.

B. Nesting decreases as dimension increases.

C. Dimension and nesting are opposite concepts that cannot coexist.

D. Dimension refers only to data types, unrelated to nesting.

Correct Answer: A

Explanation: As dimensionality increases (1-D to 2-D), the level of nested grouping (arrays within arrays) also increases correspondingly.

Q150 | Tricky | Similarities Between Dimension and Nesting

Why is understanding the similarity between dimension and nesting useful when learning about two-dimensional arrays?

A. It helps learners visualize a 2-D array as rows of one-dimensional arrays, making declaration, initialization, and access easier to understand.

B. It is not useful at all and only applies to advanced data structures beyond this syllabus.

C. It only helps with understanding functions, not arrays.

D. It replaces the need to learn indexing entirely.

Correct Answer: A

Explanation: Seeing 2-D arrays as nested 1-D arrays provides an intuitive mental model for their syntax and behavior.